TamboUI is organized as a layered library. At the bottom sits the immediate-mode API for direct terminal control. On top of that, TuiRunner adds a managed event loop. At the highest level, the Toolkit DSL provides a declarative, component-based approach.

You can use any layer directly, or mix them as needed.

API	Description	Use When
Immediate Mode	Direct terminal control - you manage the event loop, drawing, and all state	Building custom backends, game engines, or learning how things work
TuiRunner	Managed event loop with callbacks for events and rendering	Custom event handling, animations, widget-based UIs without Toolkit
Toolkit DSL	Declarative, component-based UI with fluent builders	Most applications - fastest path to a working UI
Inline Display	Fixed status area that preserves terminal scroll history	Low-level inline control, custom rendering
Inline TUI Runner	Managed event loop for inline displays with key/mouse handling	Inline apps needing event handling without full Toolkit
Inline Toolkit	Declarative element-based inline displays	Most inline applications - NPM/Gradle-style progress UIs

Immediate Mode

Direct control over the terminal backend, the buffer, and the event loop. Nothing is hidden - you decide when to poll for events, when to draw, and how to structure your main loop.

Terminal and Backend

The `Terminal` wraps a `Backend` that handles the actual terminal I/O:

```
try (var backend = new JLineBackend()) {
    var terminal = new Terminal<>(backend);
    // ...
}
```

Before you can draw anything useful, you need to set up the terminal:

```
// Raw mode disables line buffering - keys are available immediately
backend.enableRawMode();

// Alternate screen preserves the user's terminal content
backend.enterAlternateScreen();

// Hide the cursor for a cleaner look
backend.hideCursor();

// Mouse support is optional
backend.enableMouseCapture();
```

The try-with-resources pattern ensures cleanup happens automatically. If you're not using it, remember to restore the terminal state before exiting.

Drawing

All rendering goes through `terminal.draw()`:

```
terminal.draw(frame -> {
    Rect area = frame.area();  // Full terminal size

    var paragraph = Paragraph.builder()
        .text(Text.from("Hello, TamboUI!"))
        .style(Style.EMPTY.bold().fg(Color.CYAN))
        .build();

    frame.renderWidget(paragraph, area);
});
```

The `Frame` gives you:

- `area()` - the full renderable region
- `buffer()` - direct access to the cell grid
- `renderWidget()` / `renderStatefulWidget()` - render widgets to specific areas

For pixel-level control, work with the buffer directly:

```
terminal.draw(frame -> {
    Buffer buffer = frame.buffer();
    buffer.set(0, 0, new Cell("X", Style.EMPTY.bold()));

    Cell cell = buffer.get(5, 5);
    // ...
});
```

Putting It Together

```
public static class ImmediateModeExample {
    public static void main(String[] args) throws Exception {
        try (var backend = new JLineBackend()) {
            backend.enableRawMode();
            backend.enterAlternateScreen();
            backend.hideCursor();

            var terminal = new Terminal<>(backend);
```

```
            // Note: For event handling, use TuiRunner which provides
            // a managed event loop. This example shows low-level terminal setup.
            terminal.draw(frame -> {
                var widget = Paragraph.builder()
                    .text(Text.from("Press 'q' to quit"))
                    .style(Style.EMPTY.fg(Color.CYAN))
                    .build();
                frame.renderWidget(widget, frame.area());
            });
        }
    }
}
```

This is verbose, but you have complete control. For most applications, you'll want one of the higher-level APIs.

TuiRunner

TuiRunner takes over terminal setup and the event loop. You provide two callbacks: one for handling events, one for rendering.

```
try (var tui = TuiRunner.create()) {
    tui.run(
        (event, runner) -> {
            if (event instanceof KeyEvent key && key.isQuit()) {
                runner.quit();
                return false;
            }
            return handleEvent(event);
        },
        frame -> renderUI(frame)
    );
}
```

The event handler returns `true` to request a redraw, `false` otherwise. This avoids unnecessary redraws when nothing changed.

Configuration

```
var config = TuiConfig.builder()
    .mouseCapture(true)
    .tickRate(Duration.ofMillis(16))  // ~60fps for animations
    .pollTimeout(Duration.ofMillis(50))
    .build();

try (var tui = TuiRunner.create(config)) {
    tui.run(handler, renderer);
}
```

Tick events fire at the configured rate - useful for animations, clocks, or any periodic updates.

Disabling Ticks

For purely event-driven UIs that only need to refresh on user input, you can disable ticks:

```
var config = TuiConfig.builder()
    .noTick()
    .build();
```

Even with ticks disabled, the UI automatically redraws on terminal resize within the configured grace period (default 250ms). You can customize this:

```
var config = TuiConfig.builder()
    .noTick()
    .resizeGracePeriod(Duration.ofMillis(100))  // Faster resize response
    .build();
```

Set `resizeGracePeriod(null)` to disable automatic resize handling entirely (not recommended).

Error Handling

When exceptions occur during rendering, TuiRunner catches them and displays an error screen with the stack trace. By default, errors are shown in-app with options to scroll and dismiss:

```
// Default: errors display in UI, press 'q' to quit
try (var tui = TuiRunner.create()) {
    tui.run(handler, renderer);
}
```

Configure error handling behavior via `TuiConfig`:

```
// Log errors to a file and quit immediately
var config = TuiConfig.builder()
    .errorHandler(RenderErrorHandlers.logAndQuit(new PrintStream("/tmp/tui-errors.
    .build();

// Write error details to a file, then show in-app error display
var config2 = TuiConfig.builder()
    .errorHandler(RenderErrorHandlers.writeToFile(Path.of("/tmp/crash.log")))
    .build();
```

Available error handlers:

Handler	Behavior
`displayAndQuit()` (default)	Shows full-screen error display with scrollable stack trace, quit on 'q'
`logAndQuit(PrintStream)`	Logs error to stream, then exits immediately
`writeToFile(Path)`	Writes error to file, then shows in-app display
`suppress()`	Logs warning and continues (use with caution)

Custom error handlers implement `RenderErrorHandler`:

```
var config = TuiConfig.builder()
    .errorHandler((error, context) -> {
```

```
            // Log error details
            context.errorOutput().println("Error: " + error.message());
            error.cause().printStackTrace(context.errorOutput());
            // Return the action to take
            return ErrorAction.QUIT_IMMEDIATELY;
        })
        .build();
```

Threading Model

TamboUI uses a dedicated **render thread model** similar to other UI frameworks. All rendering and UI state modifications must happen on the render thread - the thread running `TuiRunner.run()`.

This model provides:

- **Thread safety**: No locks needed in UI code since everything runs on one thread
- **Predictable behavior**: UI updates happen in a well-defined order
- **Clear error reporting**: Wrong-thread access throws `IllegalStateException` with diagnostic info

The render thread

When `TuiRunner.run()` starts, it marks the current thread as the render thread. All event handling and rendering callbacks execute on this thread. When `run()` exits, the render thread is cleared.

```
// Check if on render thread
if (runner.isRenderThread()) {
    // Safe to modify UI state
}

// Or use the static utility
if (RenderThread.isRenderThread()) {
    // ...
}
```

Updating UI from Background Threads

To safely update UI state from background threads (network callbacks, timers, async tasks), use `runOnRenderThread()`:

```
// From a background thread:
runner.runOnRenderThread(() -> {
    // This runs on the render thread
    // status = "Download complete";
    // UI will redraw on next event
});
```

If called from the render thread, the action executes immediately. If called from another thread, it's queued and runs during the next event loop iteration.

For actions that should always be deferred (even when on render thread), use `runLater()`:

```
// Always queues, never executes immediately
runner.runLater(() -> {
    // Runs after current event handling completes
    processNextItem();
});
```

Scheduled Actions

Scheduled actions via `ToolkitRunner.schedule()`, `scheduleRepeating()`, and `scheduleWithFixedDelay()` run on the scheduler thread. If they modify UI state, use `runOnRenderThread()`:

```
// Action runs on scheduler thread - use runOnRenderThread for UI state
runner.schedule(() -> {
    runner.runOnRenderThread(() -> {
        // countdown--;
    });
}, Duration.ofSeconds(1));

// Same pattern for repeating actions
runner.scheduleRepeating(() -> {
```

```
    runner.runOnRenderThread(() -> animationFrame++);
}, Duration.ofMillis(16));
```

This explicit approach gives you control - you can do background work in the scheduled action and only post the UI update portion to the render thread.

Scheduler Management

Each runner maintains a single shared scheduler that handles both internal tasks (tick events, resize detection) and user-scheduled actions. By default, this scheduler is created internally when the runner starts and shut down when the runner closes.

External Scheduler Injection

For integration with frameworks that provide their own thread pools, you can inject an external scheduler via configuration:

```
ScheduledExecutorService myScheduler = Executors.newSingleThreadScheduledExecutor(

var config = TuiConfig.builder()
    .scheduler(myScheduler)  // Use externally-managed scheduler
    .build();

try (var tui = TuiRunner.create(config)) {
    tui.run(handler, renderer);
}

// myScheduler is NOT shut down when TuiRunner closes - caller retains ownership
myScheduler.shutdown();
```

Closing Semantics

When closing runners:

- **Internally-created scheduler**: Shut down automatically when the runner closes
- **Externally-provided scheduler**: NOT shut down - the caller retains ownership and is responsible for its lifecycle

This allows frameworks to manage scheduler lifecycles independently of individual runner instances.

Multiple Runners

When using multiple runners in a single application (e.g., multiple `InlineTuiRunner` instances for different inline displays), there are two approaches:

Approach	Behavior
Default (no external scheduler)	Each runner creates and owns its own scheduler. Closing one runner shuts down only its scheduler.
Shared external scheduler	All runners share the provided scheduler. Runners do not shut it down; the caller manages its lifecycle.

For multiple runners that should share a scheduler, provide an externally-managed scheduler via config to avoid premature shutdown when one runner closes.

Semantic Key Checks

`KeyEvent` provides semantic methods that respect the configured bindings:

```
if (key.isQuit()) { }      // q, Q, Ctrl+C
if (key.isUp()) { }        // Up arrow, k, Ctrl+P (with vim bindings)
if (key.isDown()) { }      // Down arrow, j, Ctrl+N
if (key.isSelect()) { }    // Enter, Space
if (key.isCancel()) { }    // Escape
if (key.isPageUp()) { }    // PageUp, Ctrl+B
if (key.isPageDown()) { }  // PageDown, Ctrl+F
```

These delegate to the bindings system - with vim bindings, `isUp()` matches `k`, with standard bindings it only matches the Up arrow.

Event Handling Patterns

Pattern matching works well with the event types:

```
EventHandler handler = (event, runner) -> {
    if (event instanceof KeyEvent k) return handleKey(k);
    if (event instanceof MouseEvent m) return handleMouse(m);
    if (event instanceof TickEvent) { animationFrame++; return true; }
    if (event instanceof ResizeEvent) return true;  // Always redraw on resize
    return false;
};
```

Layout in the Renderer

The renderer callback receives a `Frame`, just like immediate mode. Combine it with the layout system:

```
Renderer renderer = frame -> {
    var areas = Layout.vertical()
        .constraints(
            Constraint.length(3),
            Constraint.fill(),
            Constraint.length(1)
        )
        .split(frame.area());

    frame.renderWidget(header, areas.get(0));
    frame.renderWidget(content, areas.get(1));
    frame.renderWidget(statusBar, areas.get(2));
};
```

Example: Counter with Animation

```
public class CounterDemo {
    private int counter = 0;
    private int ticks = 0;

    /**
     * Runs the demo application.
     *
     * @throws Exception if an error occurs
     */
     public void run() throws Exception {
```

```
        var config = TuiConfig.builder()
            .tickRate(Duration.ofMillis(100))
            .build();

        try (var tui = TuiRunner.create(config)) {
            tui.run(this::handleEvent, this::render);
        }
    }

    private boolean handleEvent(Event event, TuiRunner runner) {
        if (event instanceof KeyEvent key && key.isQuit()) {
            runner.quit();
            return false;
        }
        if (event instanceof TickEvent) {
            ticks++;
            return true;
        }
        if (event instanceof KeyEvent key) {
            if (key.isChar('+')) { counter++; return true; }
            if (key.isChar('-')) { counter--; return true; }
        }
        return false;
    }

    private void render(Frame frame) {
        var text = String.format("Counter: %d (ticks: %d)%n%nPress +/- to change,
            counter, ticks);

        var widget = Paragraph.builder()
            .text(Text.from(text))
            .block(Block.builder()
                .title("Demo")
                .borders(Borders.ALL)
                .borderType(BorderType.ROUNDED)
                .build())
            .build();

        frame.renderWidget(widget, frame.area());
    }
}
```

TuiRunner hits a sweet spot: less boilerplate than immediate mode, but you still control the event handling and rendering logic.

Toolkit DSL

The Toolkit DSL flips the model. Instead of writing render callbacks, you describe your UI declaratively and let the framework handle the rest.

```
public static class MyApp extends ToolkitApp {

    @Override
    protected Element render() {
        return panel("My App",
            text("Hello!").bold().cyan(),
            spacer(),
            text("Press 'q' to quit").dim()
        ).rounded();
    }

    public static void main(String[] args) throws Exception {
        new MyApp().run();
    }
}
```

The static import `dev.tamboui.toolkit.Toolkit.*` gives you all the element factories: `text()`, `panel()`, `row()`, `column()`, `columns()`, `list()`, `table()`, and more.

Layout

Rows and columns work as you'd expect:

```
row(
    panel("Left").fill(),
    panel("Right").fill()
);

column(
    text("Header"),
    panel("Content").fill(),
    text("Footer")
);
```

For multi-column grid layouts, use `columns()`:

```
// Auto-detects column count based on available width and child widths
columns(item1, item2, item3, item4, item5, item6)
    .spacing(1);

// Explicit column count
columns(item1, item2, item3, item4)
    .columnCount(2);

// Column-first ordering (fills top-to-bottom, then left-to-right)
columns(item1, item2, item3, item4)
    .columnCount(2)
    .columnFirst();
```

CSS properties for columns: `column-count`, `column-order` ("row-first" or "column-first"), `spacing`, `margin`, `flex`.

Use `spacer()` to push things apart:

```
row(
    text("Left"),
    spacer(),
    text("Right")
);
```

Control positioning with flex:

```
row(
    text("Item 1"),
    text("Item 2"),
    text("Item 3")
).flex(Flex.CENTER);  // Center items horizontally

column(
    panel("Top"),
    panel("Bottom")
).flex(Flex.SPACE_BETWEEN);  // Spread items vertically
```

See Flex Layout for detailed documentation on all flex modes and usage patterns.

Styling

Style methods chain naturally:

```
text("Styled").bold().italic().cyan().onBlue();
```

Panels support border styling:

```
panel("Title", text("content"))
    .rounded()
    .borderColor(Color.CYAN)
    .focusedBorderColor(Color.YELLOW);
```

Sizing

Control how elements fill space:

```
panel("Fixed width").length(30);
panel("Take what's left").fill();
panel("Twice the weight").fill(2);
panel("At least 10").min(10);
panel("At most 50").max(50);
```

Stateful Widgets

Tables and text inputs need state objects. Lists manage their own state internally:

```
private ListElement<?> myList = list("Apple", "Banana", "Cherry")
    .highlightColor(Color.CYAN)
    .autoScroll();

private TableState tableState = new TableState();
private TextInputState inputState = new TextInputState();
```

```
Element statefulWidgetsExample() {
    // In render():
    return column(
        myList,  // ListElement manages selection and scroll internally

        table()
            .header("Name", "Age")
            .row("Alice", "30")
            .row("Bob", "25")
            .state(tableState),

        textInput(inputState)
            .placeholder("Type here...")
    );
}
```

ListElement provides methods for navigation:

```
myList.selectNext(itemCount);  // Move selection down
myList.selectPrevious();       // Move selection up
myList.selected();             // Get current selection index
myList.selected(0);            // Set selection to specific index
```

Event Handling

Attach handlers to elements:

```
panel("Interactive")
    .id("main")
    .focusable()
    .onKeyEvent(event -> {
        if (event.isChar('a')) {
            addItem();
            return EventResult.HANDLED;
        }
        return EventResult.UNHANDLED;
    });
```

Return `EventResult.HANDLED` to stop propagation, `UNHANDLED` to let it bubble up.

Focus

For Tab/Shift+Tab navigation, elements need both an ID and the focusable flag:

```
column(
    panel("First").id("first").focusable(),
    panel("Second").id("second").focusable()
);
```

The `focusedBorderColor()` method lets you provide visual feedback.

Data Visualization

```
gauge(0.75).label("Progress").gaugeColor(Color.GREEN);

sparkline(1, 4, 2, 8, 5, 7).color(Color.CYAN);

barChart(10, 20, 30)
    .barColor(Color.BLUE);
```

Wrapping Low-Level Widgets

If you have a custom widget or a widget that doesn't yet have a dedicated Toolkit element, you can wrap it with `widget()`:

```
// Wrap any Widget for use in the Toolkit DSL
widget(myCustomWidget)
    .addClass("custom")
    .fill();

// Use in layouts like any other element
row(
    widget(customWidget1).fill(),
```

```
        widget(customWidget2).fill()
    );
```

This is only useful when:

- You have a custom widget without a dedicated element wrapper
- You want quick integration of a widget into the Toolkit DSL
- You need to apply layout constraints, CSS classes, or event handlers to a widget

Limitations:

- **Styling does not propagate** - Colors and modifiers set on the element don't affect the widget's internal rendering. The widget renders directly to the buffer using its own styling logic.
- **No CSS child selectors** - GenericWidgetElement has no sub-components that can be styled via CSS.
- **No preferred size** - The element doesn't know the widget's size requirements, so the container must specify constraints.

For full styling support, consider creating a dedicated element wrapper for your widget type (see `TextElement` or `GaugeElement` as examples).

Using ToolkitRunner Directly

`ToolkitApp` is convenient, but `ToolkitRunner` gives more control:

```
var config = TuiConfig.builder()
    .mouseCapture(true)
    .tickRate(Duration.ofMillis(50))
    .build();

try (var runner = ToolkitRunner.create(config)) {
    runner.run(() -> panel("App", content));
}
```

Fault-Tolerant Rendering

When enabled, fault-tolerant mode catches exceptions from individual elements and displays error placeholders instead of crashing the entire application. The rest of the UI continues to render normally.

```
try (var runner = ToolkitRunner.builder()
        .faultTolerant(true)
        .build()) {
    runner.run(() -> render());
}
```

With fault-tolerant mode:

- If an element throws during `render()`, an error placeholder is displayed in its area
- The error placeholder shows a red border with the exception type and message
- Other elements continue to render normally
- Useful for dashboards where one failing widget shouldn't break the entire UI

Without fault-tolerant mode (default):

- Exceptions propagate to `TuiRunner`, which displays a full-screen error
- This is the safer default for most applications

Example: Todo List

```
public static class TodoApp extends ToolkitApp {
    private final List<String> items = new ArrayList<>(List.of(
        "Learn TamboUI",
        "Build something cool"
    ));
    private final ListElement<?> todoList = list()
        .highlightColor(Color.CYAN)
        .autoScroll();

    @Override
    protected Element render() {
        return panel("Todo",
            items.isEmpty()
                ? text("Empty - press 'a' to add").dim()
```

```
                : todoList.items(items.toArray(new String[0])),
            spacer(),
            text("[a]dd [d]elete [q]uit").dim()
        )
        .rounded()
        .id("main")
        .focusable()
        .onKeyEvent(this::handleKey);
    }

    private EventResult handleKey(KeyEvent event) {
        if (event.isChar('a')) {
            items.add("New item");
            return EventResult.HANDLED;
        }
        if (event.isChar('d') && !items.isEmpty()) {
            items.remove(todoList.selected());
            return EventResult.HANDLED;
        }
        if (event.isDown()) {
            todoList.selectNext(items.size());
            return EventResult.HANDLED;
        }
        if (event.isUp()) {
            todoList.selectPrevious();
            return EventResult.HANDLED;
        }
        return EventResult.UNHANDLED;
    }

    public static void main(String[] args) throws Exception {
        new TodoApp().run();
    }
}
```

Inline Display Mode

The APIs above all use the alternate screen buffer - your application takes over the entire terminal, and the previous content is restored when it exits.

For CLI tools that need to show progress while preserving scroll history, use `InlineDisplay` instead. This is the pattern used by Gradle, npm, and other build tools: a fixed status area at the

bottom while log output scrolls above.

When to Use Inline Mode

- Build tools showing compilation progress
- Package managers displaying download status
- Long-running scripts with progress indicators
- Any tool where you want output history preserved

Basic Usage

```
try (var display = InlineDisplay.create(3)) {  // Reserve 3 lines
    for (int i = 0; i <= 100; i += 10) {
        final int progress = i;
        display.render((area, buffer) -> {
            var gauge = Gauge.builder()
                .ratio(progress / 100.0)
                .label("Processing: " + progress + "%")
                .build();
            gauge.render(area, buffer);
        });
        Thread.sleep(100);
    }
}
```

The display reserves lines at the current cursor position. When closed, it moves the cursor below and optionally clears the status area.

Logging Above the Status Area

Use `println()` to add output that scrolls above the fixed status:

```
try (var display = InlineDisplay.create(4)) {
    for (var task : tasks) {
        // Update status area
        display.render((area, buffer) -> {
            renderProgress(area, buffer, task, progress);
        });
```

```
        processTask(task);

        // Log completion - scrolls above, status stays fixed
        display.println(Text.from(Line.from(
            Span.styled("OK ", Style.EMPTY.fg(Color.GREEN)),
            Span.raw(task.name())
        )));
    }
}
```

Configuration Options

```
// Fixed height (width matches terminal)
var display = InlineDisplay.create(4);

// Fixed height and width
var display2 = InlineDisplay.create(4, 80);

// Clear the status area when done
var display3 = InlineDisplay.create(4).clearOnClose();
```

Setting Lines Directly

For simple status updates, set lines without a full render:

```
display.setLine(0, "Building module: core");
display.setLine(1, "Progress: 45%");
display.setLine(2, Text.from(Span.styled("No errors", Style.EMPTY.fg(Color.GREEN))
```

Inline vs Alternate Screen

Feature	InlineDisplay	TuiRunner / Toolkit
Screen buffer	Main buffer (preserved)	Alternate buffer (replaced)

Feature	InlineDisplay	TuiRunner / Toolkit
Cursor	Visible	Hidden
Terminal capture	Partial (reserved lines)	Full terminal
Previous content	Preserved above	Restored on exit
Best for	Progress, logs, status	Interactive full-screen apps

Inline TUI Runner

`InlineTuiRunner` adds a managed event loop to inline displays, similar to how `TuiRunner` works for full-screen apps. It handles keyboard and mouse events, tick-based animations, and thread-safe updates.

Basic Usage

```
try (var runner = InlineTuiRunner.create(4)) {
    runner.run(
        // Event handler
        (event, r) -> {
            if (event instanceof KeyEvent key && key.character() == 'q') {
                r.quit();
                return true;
            }
            return false;
        },
        // Renderer
        frame -> {
            var gauge = Gauge.builder()
                .ratio(progress / 100.0)
                .build();
            gauge.render(frame.area(), frame.buffer());
        }
```

```
    );
}
```

Configuration

```
var config = InlineTuiConfig.builder(4)  // 4 lines
    .tickRate(Duration.ofMillis(50))     // For animations
    .clearOnClose(true)                   // Clear viewport on exit
    .build();

try (var runner = InlineTuiRunner.create(config)) {
    // ...
}
```

Printing Above the Viewport

```
// Plain text
runner.println("Task completed!");

// Styled text
runner.println(Text.from(Line.from(
    Span.styled("OK ", Style.EMPTY.fg(Color.GREEN)),
    Span.raw("Build successful")
)));
```

Thread-Safe Updates

```
// Run code on the render thread
runner.runOnRenderThread(() -> {
    // progress = newValue;
});

// Schedule for later
runner.runLater(() -> {
    cleanup();
});
```

Inline Toolkit

The Inline Toolkit provides the same declarative, element-based API as the full Toolkit DSL, but for inline displays. This is the recommended approach for most inline applications.

InlineApp Base Class

The easiest way to create an inline application:

```java
public static class MyProgressApp extends InlineApp {

    private double progress = 0.0;

    public static void main(String[] args) throws Exception {
        new MyProgressApp().run();
    }

    @Override
    protected int height() {
        return 3;  // Reserve 3 lines
    }

    @Override
    protected Element render() {
        return column(
            text("Installing packages...").bold(),
            gauge(progress).green(),
            text(String.format("%.0f%% complete", progress * 100)).dim()
        );
    }

    @Override
    protected void onStart() {
        // Schedule periodic updates
        runner().scheduleRepeating(() -> {
            runner().runOnRenderThread(() -> {
                progress += 0.01;
                if (progress >= 1.0) quit();
            });
        }, Duration.ofMillis(50));
    }
}
```

Configuration

Override `configure()` to customize behavior:

```
public static class MyConfiguredInlineApp extends InlineApp {
    @Override
    protected int height() { return 3; }

    @Override
    protected Element render() { return text(""); }

    @Override
    protected InlineTuiConfig configure(int height) {
        return InlineTuiConfig.builder(height)
            .tickRate(Duration.ofMillis(30))  // Faster for smooth animations
            .clearOnClose(false)               // Keep output visible
            .build();
    }
}
```

Printing Elements

Print styled content above the viewport:

```
// Print a styled element
println(row(
    text("OK ").green().fit(),
    text("lodash").fit(),
    text("@4.17.21").dim().fit()
).flex(Flex.START));

// Print plain text
println("Step completed");
```

Handling Key Events

Use focusable elements or global key handlers:

```java
public static class MyInteractiveInlineApp extends InlineApp {
    @Override
    protected int height() { return 3; }

    @Override
    protected Element render() {
        return column(
            text("Continue? [Y/n]").cyan(),
            spacer()
        )
        .focusable()
        .onKeyEvent(event -> {
            if (event.character() == 'y' || event.character() == 'Y') {
                startNextPhase();
                return EventResult.HANDLED;
            } else if (event.character() == 'n') {
                quit();
                return EventResult.HANDLED;
            }
            return EventResult.UNHANDLED;
        });
    }

    void startNextPhase() {}
}
```

Text Input in Inline Mode

Text inputs work with proper focus management:

```java
public static class MyFormInlineApp extends InlineApp {
    private final TextInputState nameState = new TextInputState();

    @Override
    protected int height() { return 3; }

    @Override
    protected Element render() {
        return column(
            row(
                text("Name: ").bold().fit(),
                textInput(nameState)
```

```
                    .id("name-input")
                    .placeholder("Enter name...")
                    .constraint(Constraint.length(20))
                    .onSubmit(() -> handleSubmit(nameState.text()))
            ).flex(Flex.START),
            text("[Enter] Submit  [Tab] Next field").dim()
        );
    }

    private void handleSubmit(String value) {}
}
```

Using InlineToolkitRunner Directly

For more control, use `InlineToolkitRunner` with a render function:

```
try (var runner = InlineToolkitRunner.create(3)) {
    runner.run(() -> column(
        waveText("Processing...").cyan(),
        gauge(progress),
        text("Please wait").dim()
    ));
}
```

Scopes for Dynamic Regions

Use `scope()` from `InlineToolkit` to create collapsible regions:

```
public static class MyScopedInlineApp extends InlineApp {
    private boolean downloading = true;
    private double progress1 = 0;
    private double progress2 = 0;

    @Override
    protected int height() { return 5; }

    @Override
    protected Element render() {
        return column(
            text("Package Installation").bold(),
```

```
            // This section collapses when downloading becomes false
            scope(downloading,
                row(text("file1.zip: "), gauge(progress1)),
                row(text("file2.zip: "), gauge(progress2))
            ),

            text(downloading ? "Downloading..." : "Complete!").dim()
        );
    }
}
```

When `downloading` becomes `false`, the scope collapses to zero height and the layout adjusts automatically.

Inline vs Full-Screen Toolkit

Feature	InlineApp / InlineToolkitRunner	App / ToolkitRunner
Screen	Inline (preserves scroll)	Alternate screen (full takeover)
Height	Fixed (specified lines)	Dynamic (full terminal)
Output	`println()` scrolls above	No scrolling output
Use case	Progress, status, forms	Full interactive applications

Choosing a Level

Start with the Toolkit DSL for most applications. It handles focus, provides a clean declarative API, and gets you productive quickly.

Drop to TuiRunner when you need custom event handling, animations, or want to use the widget layer directly without the Toolkit abstractions.

Use Immediate Mode when you're building something unusual - a custom backend, a game engine, or when you want to understand exactly what's happening under the hood.

The levels compose well. You can use Toolkit elements inside a TuiRunner application, or drop down to direct buffer manipulation when needed.

Next Steps

- Bindings and Actions - key bindings and action handling
- CSS Styling - external stylesheets and theming
- Application Structure - patterns for larger applications
- Widgets Reference - all available widgets
- Developer Guide - building custom widgets

PREVIOUS
Demo Gallery

NEXT
CSS Styling